

Computer Architecture (25VBT0C104)



www.onlinejain.com

Module: 03

Advanced Memory & Arithmetic Units

Module Information

Field	Details
Course Title	Computer Architecture
Course Code	25VBT0C104
Module Number	Module 3 of 5
Module Title	Advanced Memory and Arithmetic Units
Semester	I
Credits	4

Module Overview

Module 3 of Computer Architecture ventures into two critical pillars of modern computing: the memory subsystem and arithmetic hardware. As programs grow in complexity, the gap between processor speed and memory speed becomes a significant performance bottleneck. This module begins by exploring how computer architects address this challenge through cache memory — a high-speed memory layer placed between the processor and main memory.

You will study how cache is organised, how data is mapped from main memory to cache using direct, associative, and set-associative techniques, and how replacement algorithms determine which data to retain when the cache is full. Strategies to optimise cache performance, including write policies and cache coherence, are also covered.

The module then transitions to virtual memory — an elegant abstraction that enables programs to use more memory than physically available. Address translation mechanisms, page tables, and the role of the Memory Management Unit (MMU) are examined in depth.

The second half of the module shifts focus to the digital logic foundations of arithmetic hardware. Starting with fundamental building blocks — logic gates and flip-flops — you will understand how binary addition and subtraction are implemented at the hardware level. Particular attention is given to the challenges of carry propagation and the innovative carry look-ahead adder design that accelerates computation in modern processors.

By the end of this module, you will be equipped to analyse memory design trade-offs and apply knowledge of arithmetic circuits to evaluate processor designs.

Learning Objectives

By the end of this module, students will be able to:

- Apply knowledge of cache memory organisation to classify mapping techniques and their trade-offs.
- Analyse cache replacement algorithms and evaluate their effectiveness in different scenarios.
- Apply the concept of virtual memory to explain address translation and memory management.
- Analyse the design of arithmetic circuits, including logic gates and flip-flops, in the context of binary operations.
- Apply fast adder design principles, specifically carry look-ahead, to solve arithmetic computation problems.

Learning Outcomes

Upon successful completion of this module, students will be able to:

- Differentiate between direct-mapped, fully associative, and set-associative cache and apply appropriate mapping for given scenarios.
- Evaluate cache replacement policies (LRU, FIFO, LFU) and analyse their impact on hit rates and system performance.
- Demonstrate how virtual memory works through address translation using page tables and TLBs.
- Analyse the behaviour of logic gates and flip-flops in the construction of binary arithmetic circuits.
- Design and analyse carry look-ahead adder circuits and compare their performance against ripple carry adders.

Table of Topics

Sub-Unit	Topic	Coverage
3.1	Memory Hierarchy — An Overview	Levels, speed vs cost, introduction to cache
3.2	Cache Memory Organisation	Cache structure, lines, tags, hits and misses
3.3	Cache Mapping Techniques	Direct mapped, fully associative, set-associative
3.4	Cache Replacement Algorithms	LRU, FIFO, LFU, random replacement
3.5	Cache Performance Optimisation	Write policies, miss rate reduction, cache coherence
3.6	Virtual Memory and Address Translation	Paging, page tables, MMU, TLB
3.7	Logic Gates and Flip-Flops	AND, OR, NOT, XOR, SR/D/JK flip-flops
3.8	Addition and Subtraction of Signed Numbers	2's complement, overflow detection, subtraction circuits
3.9	Fast Adder Designs — Carry Look-Ahead Adder	Ripple carry limitations, CLA concept, generate and propagate

3.1 Memory Hierarchy — An Overview

Modern computers must balance the competing demands of speed, capacity, and cost in their memory systems. Ideally, every computer would have a single, infinitely large memory that is also blazingly fast and inexpensive — but in practice, no such technology exists. Faster memory technologies are more expensive and physically smaller, while larger memories tend to be slower and cheaper.

Computer architects address this fundamental conflict by organising memory into a hierarchy — multiple layers of storage with different speed, size, and cost characteristics. The key insight that makes this hierarchy practical is the principle of locality of reference.

The Principle of Locality

Programs do not access all memory locations equally. Two patterns dominate:

- **Temporal Locality:** If a memory location is accessed, it is likely to be accessed again soon. For example, a loop counter variable is used repeatedly during loop execution.
- **Spatial Locality:** If a memory location is accessed, nearby locations are likely to be accessed soon. For instance, scanning through an array accesses sequential memory addresses.

The memory hierarchy exploits these patterns by keeping recently and frequently accessed data in faster, smaller, and costlier memory levels.

Levels of the Memory Hierarchy

Level	Type	Speed	Size	Cost per GB
L1 Cache	SRAM	< 1 ns	32–64 KB	Very High
L2/L3 Cache	SRAM	2–10 ns	256 KB – 32 MB	High
Main Memory (RAM)	DRAM	50–100 ns	4 – 128 GB	Moderate
SSD Storage	Flash	50–150 μ s	128 GB – 4 TB	Low
HDD Storage	Magnetic	5–10 ms	500 GB – 20 TB	Very Low

💡 Did You Know?

India's semiconductor memory market is projected to exceed USD 20 billion by 2030, driven by domestic manufacturing initiatives under the India Semiconductor Mission (ISM) launched in 2021. Companies like Tata Electronics and Micron Technology have announced major chip fabrication investments in Gujarat and Assam respectively.

3.2 Cache Memory Organisation

Cache memory is a small, ultra-fast memory bank that sits between the CPU and main memory (RAM). Its purpose is to reduce the effective memory access time by keeping a copy of frequently accessed data close to the processor. When the CPU needs to read data, it first checks the cache — if found, the access is served instantly from cache; if not, it fetches from the slower main memory.

Key Terminology

- **Cache Line (Block):** The fundamental unit of data transfer between cache and main memory. A typical cache line is 64 bytes. Data is moved in entire lines, not individual bytes.
- **Cache Hit:** When the CPU finds the requested data in the cache. Hit time is typically 1–4 clock cycles.
- **Cache Miss:** When the requested data is NOT in the cache, requiring a fetch from main memory (50–200+ clock cycles).
- **Hit Rate (h):** The fraction of memory accesses that result in a cache hit. Hit Rate = Hits / (Hits + Misses).
- **Miss Rate:** The complement of hit rate. Miss Rate = 1 – Hit Rate.
- **Tag:** A portion of the memory address stored in the cache to identify which main memory block occupies a given cache slot.
- **Valid Bit:** A single bit per cache line that indicates whether the data in that cache slot is valid and usable.

Effective Memory Access Time

The average time to access memory with a cache is calculated as:

$$\text{Effective Access Time} = \text{Hit Rate} \times \text{Cache Time} + \text{Miss Rate} \times \text{Main Memory Time}$$

Example: Suppose cache access time = 5 ns, main memory access time = 100 ns, hit rate = 90%.

- Effective Access Time = $0.90 \times 5 + 0.10 \times 100 = 4.5 + 10 = 14.5 \text{ ns}$
- Without cache, every access would take 100 ns — the cache reduces average access time by $\sim 6.9\times$.

🔍 Analysing Misconceptions: Cache Size and Performance

✗ MYTH: A larger cache always means faster performance, and doubling cache size doubles speed.

✓ REALITY: Beyond a certain size, the returns from increasing cache diminish sharply. A larger cache also has higher latency for lookup operations. Additionally, cache effectiveness depends far more on the hit rate and access pattern than on absolute size.

3.3 Cache Mapping Techniques

A fundamental design question in cache architecture is: given a block of data from main memory, where exactly can it be stored in the cache? The answer defines the mapping technique used. There are three primary approaches, each representing a different trade-off between hardware complexity and flexibility.

3.3.1 Direct-Mapped Cache

In a direct-mapped cache, each block of main memory can map to exactly one specific line in the cache. The cache line number is determined by the formula:

Cache Line Number = (Main Memory Block Number) mod (Total Cache Lines)

The memory address is split into three fields: Tag | Line Index | Block Offset.

- Tag: Upper bits of the address — identifies which of the many main memory blocks currently occupies this cache line.
- Line Index: Middle bits — selects the specific cache line.

- Block Offset: Lower bits — selects the specific byte within the cache line.

Example

Consider a cache with 8 lines. Memory block 0 → Line 0, block 1 → Line 1, ..., block 8 → Line 0 again, block 9 → Line 1, and so on. If a program accesses blocks 0 and 8 alternately, they both compete for Line 0, causing continuous misses — a phenomenon known as cache thrashing.

- Advantage: Simple hardware, very fast lookup.
- Disadvantage: Conflict misses when multiple memory blocks compete for the same cache line.

3.3.2 Fully Associative Cache

In fully associative cache, any block of main memory can be placed into any available cache line. There is no fixed mapping — a block is simply loaded into whichever line is free (or chosen by the replacement algorithm).

The memory address is split into two fields: Tag | Block Offset.

- The processor must search ALL cache lines simultaneously to check if the data is present — this requires comparator circuits for every line, which becomes expensive as cache size grows.
- Advantage: No conflict misses — maximum flexibility in block placement.
- Disadvantage: High hardware cost and complexity; search time increases with cache size.

3.3.3 Set-Associative Cache

Set-associative cache is the most widely used compromise. The cache is divided into a number of sets, and each set contains a fixed number of lines (called "ways"). A main memory block can be placed in any line within its designated set.

A k-way set-associative cache means each set contains k lines. The memory address is split as: Tag | Set Index | Block Offset.

- 2-Way Set Associative: 2 lines per set — balances flexibility and hardware cost.
- 4-Way Set Associative: Used in most modern L2/L3 caches.
- When k equals the total number of lines: behaves like fully associative. When $k = 1$: behaves like direct-mapped.

Technique	Key Characteristics
Direct Mapped	Each block → exactly one cache line. Simple, fast, prone to conflict misses.
Fully Associative	Any block → any line. Maximum flexibility, expensive parallel search.
Set Associative (k-way)	Block → any line within a fixed set. Best balance; used in most modern processors.

💡 Did You Know?

The Apple M-series chips (M1, M2, M3) use up to 16 MB of L2 cache per cluster and a large shared "system-level cache" (SLC) to bridge the gap between L2 and unified memory. This aggressive cache hierarchy contributes significantly to their industry-leading energy efficiency.

 Let's Check

Q.No	Type	Question
1	MCQ	In a direct-mapped cache with 16 lines, which line does memory block number 25 map to? (A) Line 7 (B) Line 9 (C) Line 25 (D) Line 8
2	MCQ	Which cache mapping technique uses parallel comparators for all cache lines? (A) Direct Mapped (B) Set Associative (C) Fully Associative (D) Paged
3	Fill in Blank	In a set-associative cache, the cache is divided into _____, each containing a fixed number of ways.
4	Fill in Blank	The _____ field of a memory address in direct-mapped cache identifies which main memory block currently occupies a given cache line.
5	True/False	Fully associative cache eliminates conflict misses entirely. (True/False)
6	True/False	A 1-way set-associative cache behaves the same as a fully associative cache. (True/False)

Answers:

Q.No	Answer
1	(B) Line 9 ($25 \bmod 16 = 9$)
2	(C) Fully Associative
3	Sets
4	Tag
5	True
6	False — A 1-way set-associative cache behaves like a direct-mapped cache. When $k = \text{total lines}$, it behaves like fully associative.

3.4 Cache Replacement Algorithms

In set-associative and fully associative caches, when a new block must be loaded and the target set (or cache) is full, one of the existing blocks must be evicted. The replacement algorithm determines which block to remove. The goal is to retain blocks that are most likely to be needed again — minimising future misses.

3.4.1 Least Recently Used (LRU)

LRU evicts the block that has not been accessed for the longest time. It operates on the principle of temporal locality: if a block has not been used recently, it is less likely to be needed in the near future.

- Implementation: A counter or timestamp is maintained for each cache line. On every access, the counter is updated. When replacement is needed, the line with the oldest timestamp is evicted.

- Performance: LRU generally performs very well because it closely approximates the "optimal" replacement policy.
- Hardware Cost: Can be expensive to implement in hardware for large caches; approximations like "pseudo-LRU" are commonly used.

Illustration: Assume a 3-line cache (fully associative) and the following access sequence: A, B, C, D, A, B, E, A, B, C, D, E.

- After loading A, B, C: cache = [A, B, C]
- Access D (miss): Evict A (least recently used) → cache = [B, C, D]
- Access A (miss): Evict B → cache = [C, D, A]

3.4.2 First-In, First-Out (FIFO)

FIFO evicts the block that has been in the cache the longest, regardless of how recently it was accessed. The cache behaves like a queue — the oldest arrival is the first to leave.

- Advantage: Simple to implement — just maintain the order of arrival.
- Disadvantage: May evict frequently used blocks if they were loaded early. Does not account for temporal locality.
- Belady's Anomaly: Increasing cache size can sometimes increase the miss rate with FIFO — a counterintuitive phenomenon that does not affect LRU.

3.4.3 Least Frequently Used (LFU)

LFU evicts the block with the fewest accesses over its lifetime in the cache. The assumption is that blocks accessed most often are most valuable.

- Advantage: Effective for workloads where access frequency is stable and predictable.
- Disadvantage: A block loaded long ago with a high historical count may retain its position even if it is no longer relevant — this 'cache pollution' can degrade performance.

3.4.4 Random Replacement

A random cache line within the target set is selected for eviction, with no consideration of access history.

- Advantage: Extremely simple hardware implementation; avoids worst-case patterns that can plague deterministic algorithms.
- Disadvantage: Average performance is typically lower than LRU.
- Surprising fact: In practice, random replacement often performs within 5–10% of LRU while requiring almost no hardware overhead — making it attractive for embedded systems and energy-constrained devices.

🔍 Analysing Misconceptions: Cache Replacement and Miss Rate

✗ MYTH: LRU is always the best cache replacement policy in all scenarios.

✓ REALITY: LRU is optimal for workloads with high temporal locality, but for workloads with cyclic or streaming access patterns (e.g., scanning a large array larger than cache), LRU performs poorly — all accesses result in misses. Alternative policies like FIFO or random can outperform LRU in specific access patterns.

3.5 Cache Performance Optimisation

Building an effective cache is not just about choosing the right mapping and replacement policy — write operations, multi-level hierarchies, and data consistency all play important roles. This unit examines the key strategies used to maximise cache performance.

3.5.1 Write Policies

When a processor writes data, the cache must decide how to handle updates to the corresponding main memory copy. Two primary policies govern this:

Write-Through

Every write to the cache is immediately propagated to main memory as well. The cache and main memory are always consistent.

- Advantage: Simple implementation; main memory is always up to date.
- Disadvantage: Every write generates a memory bus transaction, creating a significant bandwidth bottleneck.
- Solution: A write buffer is often used to temporarily hold writes before committing them to main memory, masking the latency.

Write-Back

Writes are made only to the cache. A dirty bit is set to indicate that the cache line differs from main memory. The updated data is written back to main memory only when the cache line is evicted.

- Advantage: Dramatically reduces memory bus traffic — multiple writes to the same location result in only one eventual write to memory.
- Disadvantage: Main memory may be stale; coherence issues can arise in multiprocessor systems.

3.5.2 Categories of Cache Misses (The Three Cs)

- Compulsory Misses (Cold Misses): Occur the very first time a block is accessed — unavoidable, as the data has never been in cache.
- Capacity Misses: Occur because the cache is not large enough to hold all the data needed by the program. Increasing cache size reduces these.
- Conflict Misses: Occur in direct-mapped or set-associative caches when multiple blocks compete for the same cache line/set. Increasing associativity reduces these.

3.5.3 Multi-Level Cache Hierarchy

Modern processors incorporate multiple levels of cache to balance speed and size:

- L1 Cache: Closest to the CPU core, smallest (32–64 KB), fastest (1–4 cycles). Typically split into separate instruction cache (I-cache) and data cache (D-cache).
- L2 Cache: Unified (instructions + data), larger (256 KB – 8 MB), slightly slower (5–15 cycles). Acts as backup for L1.
- L3 Cache: Shared across all CPU cores, very large (8–64 MB), slower than L2 but still far faster than RAM. Critical for multi-core systems.

💡 Did You Know?

Intel's 13th generation Core processors (Raptor Lake, 2022) include up to 36 MB of L3 cache — a result of Intel's shift to a hybrid big.LITTLE architecture combining performance cores and efficiency cores. AMD's Ryzen 7000 series features 3D V-Cache technology that stacks an additional 64 MB of L3 directly on top of the compute die using TSV (Through Silicon Via) technology.

 Let's Check

Q.No	Type	Question
1	MCQ	Which write policy sets a 'dirty bit' and writes to main memory only on eviction? (A) Write-Through (B) Write-Back (C) Write-Allocate (D) Write-Buffer
2	MCQ	Which category of cache miss occurs the very first time a memory block is accessed? (A) Conflict Miss (B) Capacity Miss (C) Compulsory Miss (D) Coherence Miss
3	Fill in Blank	The three Cs of cache misses are: Compulsory, Capacity, and _____ misses.
4	Fill in Blank	In a multi-level cache hierarchy, the _____ cache is shared across all CPU cores and is largest in size.
5	True/False	Write-through policy causes more memory bus traffic than write-back policy. (True/False)
6	True/False	Conflict misses can be reduced by increasing the associativity of the cache. (True/False)

Answers:

Q.No	Answer
1	(B) Write-Back
2	(C) Compulsory Miss
3	Conflict

4	L3
5	True
6	True

3.6 Virtual Memory and Address Translation

Virtual memory is one of the most elegant abstractions in computer science. It creates the illusion that each process has access to a large, contiguous block of memory — even if the physical RAM is smaller or fragmented. The operating system and hardware collaborate to map virtual addresses used by programs to actual physical addresses in RAM.

This abstraction provides three key benefits:

- **Process Isolation:** Each process operates in its own virtual address space. A bug in one program cannot directly corrupt another program's memory.
- **Memory Overcommitment:** Programs can be larger than physical RAM; pages not currently in use are stored on disk (swap space) and loaded on demand.
- **Simplified Programming:** Programmers write code without knowledge of physical memory layout; the OS handles allocation transparently.

3.6.1 Paging

In paging, both virtual address space and physical memory are divided into fixed-size blocks. Virtual blocks are called pages; physical blocks are called frames. Pages and frames are the same size (typically 4 KB).

A page table maintained by the OS maps virtual page numbers (VPNs) to physical frame numbers (PFNs). When a process accesses a virtual address:

- **Step 1:** The MMU extracts the Virtual Page Number (VPN) and Page Offset from the virtual address.

- Step 2: The MMU looks up the VPN in the page table to find the corresponding Physical Frame Number (PFN).
- Step 3: The physical address is formed as: PFN concatenated with the Page Offset.
- Page Fault: If the page table entry marks the page as not present in RAM, a page fault exception is raised. The OS loads the required page from disk into a free frame and updates the page table.

3.6.2 Translation Lookaside Buffer (TLB)

The page table is stored in main memory — looking it up on every memory access would double the memory access time. The Translation Lookaside Buffer (TLB) is a small, fast hardware cache (typically 32–1024 entries) that stores recent virtual-to-physical address translations.

- TLB Hit: The translation is found in the TLB — physical address is obtained in 1–2 cycles (same as cache lookup).
- TLB Miss: Translation is not in TLB — the MMU performs a page table walk (100+ cycles) and loads the entry into the TLB.
- TLB Reach: The total memory covered by TLB entries. A TLB with 512 entries $\times$ 4 KB pages covers 2 MB of virtual address space.
- TLB Flush: On a context switch (switching between processes), TLB entries for the old process must be invalidated, as virtual addresses now refer to a different process.

3.6.3 Page Replacement

When a page fault occurs and RAM is full, the OS must select a page to evict to disk. Page replacement algorithms mirror cache replacement policies:

- Optimal (OPT): Evict the page that will not be used for the longest time in the future. Theoretical — requires future knowledge.

- LRU: Evict the least recently used page. Practical approximation of optimal; used in Linux through an active/inactive page list mechanism.
- Clock Algorithm (Second Chance): An efficient approximation of LRU that uses a reference bit and a "clock hand" pointer to select victims.

💡 Did You Know?

India's PARAM Siddhi-AI supercomputer (operated by C-DAC, Pune) uses advanced virtual memory management across its 210 NVIDIA A100 GPUs to provide a unified address space for deep learning workloads. The system is ranked among the top 100 supercomputers globally (Top500 list, 2022) and supports India's National AI Mission.

3.7 Logic Gates and Flip-Flops

All digital circuits — from simple calculators to complex processors — are built from elementary logic gates. Understanding these building blocks is essential to grasping how computers perform arithmetic operations in hardware. This unit provides a concise but thorough treatment of logic gates and the fundamental memory element: the flip-flop.

3.7.1 Basic Logic Gates

Gate	Boolean Expression	Behaviour
AND	$Y = A \cdot B$	Output is 1 only if ALL inputs are 1.
OR	$Y = A + B$	Output is 1 if ANY input is 1.
NOT	$Y = \bar{A}$ (A-bar)	Output is the complement (inverse) of input.
NAND	$Y = (A \cdot B)^{\bar{}}$	Output is 0 only if ALL inputs are 1. NAND is 'universal' — all other gates can be built from it.
NOR	$Y = (A + B)^{\bar{}}$	Output is 1 only if ALL inputs are 0. NOR is also a universal gate.
XOR	$Y = A \oplus B$	Output is 1 if inputs DIFFER. Critical for addition circuits.
XNOR	$Y = (A \oplus B)^{\bar{}}$	Output is 1 if inputs are EQUAL. Used in comparator circuits.

The XOR gate is particularly important for arithmetic circuits because it implements the sum bit of binary addition: $0 \text{ XOR } 0 = 0$, $0 \text{ XOR } 1 = 1$, $1 \text{ XOR } 0 = 1$, $1 \text{ XOR } 1 = 0$ (sum is 0, carry is 1).

3.7.2 Half Adder and Full Adder

Before introducing flip-flops, it is helpful to see how gates combine to perform binary addition:

- Half Adder: Adds two single bits (A and B). Produces a Sum ($S = A \text{ XOR } B$) and a Carry ($C = A \text{ AND } B$). Cannot handle a carry-in from a previous bit position.

- Full Adder: Adds three bits — A, B, and Carry-In (Cin). Produces Sum and Carry-Out. Full adders are chained together to build multi-bit adders.

Full Adder equations:

- $\text{Sum} = A \oplus B \oplus \text{Cin}$
- $\text{Carry-Out} = (A \text{ AND } B) \text{ OR } (B \text{ AND } \text{Cin}) \text{ OR } (A \text{ AND } \text{Cin})$

3.7.3 Flip-Flops — Digital Memory Elements

Combinational circuits (made of logic gates) have no memory — their output depends only on the current inputs. Sequential circuits use flip-flops, which can store one bit of information and change state on a clock signal. Flip-flops are the fundamental building blocks of registers, counters, and state machines.

SR (Set-Reset) Flip-Flop

The simplest sequential element. Two inputs: S (Set) and R (Reset). Setting $S=1$ makes $Q=1$; $R=1$ makes $Q=0$. The condition $S=R=1$ is invalid (ambiguous output).

D (Data) Flip-Flop

The D flip-flop has a single data input D and captures its value at the clock edge. $Q = D$ when clock triggers. The D flip-flop is the most common type in register and memory design — it is the building block of CPU registers in processors like ARM's Cortex-A series.

JK Flip-Flop

An improvement over SR — the condition $J=K=1$ is valid and causes the output to toggle (invert). The JK flip-flop is widely used in counters and frequency divider circuits.

- $J=0, K=0$: No change (hold state)
- $J=1, K=0$: Set ($Q = 1$)
- $J=0, K=1$: Reset ($Q = 0$)
- $J=1, K=1$: Toggle (Q flips)

💡 Did You Know?

The Intel 4004 — the world's first commercial microprocessor (1971) — contained 2,300 transistors and used simple PMOS logic gates. Today's Apple M3 Ultra chip packs 184 billion transistors. This 80-million-fold increase in 50 years is a testament to Moore's Law in action.

3.8 Addition and Subtraction of Signed Numbers

Computers represent negative numbers using a system called 2's complement, which is the universal standard for signed integer arithmetic in modern processors. Understanding this representation is essential for analysing how CPUs perform arithmetic operations.

3.8.1 Representing Signed Numbers: 2's Complement

In an n-bit 2's complement system:

- Positive numbers are represented in the standard binary form (with the most significant bit, MSB, being 0).
- Negative numbers have MSB = 1 and their value is obtained by inverting all bits and adding 1.
- Range: For n bits, the range is $-2^{(n-1)}$ to $+2^{(n-1)} - 1$. For 8-bit: -128 to +127.

Example (8-bit): To represent -37:

- Step 1: Write +37 in binary: 00100101
- Step 2: Invert all bits (1's complement): 11011010
- Step 3: Add 1: 11011011 ← This is -37 in 2's complement.

3.8.2 Addition of Signed Numbers

The beauty of 2's complement is that addition works the same way for both positive and negative numbers — the same hardware (adder) is used.

Example 1: $(+5) + (+3) = +8$

- $00000101 + 00000011 = 00001000$ ✓ (MSB = 0, positive result)

Example 2: $(+5) + (-3) = +2$

- $00000101 + 11111101 = 100000010$
- Discard carry-out from the 8th bit: $00000010 = +2$ ✓

Example 3: $(-5) + (-3) = -8$

- $11111011 + 11111101 = 11111000$
- Discard carry-out: $11111000 = -8$ ✓

3.8.3 Subtraction Using 2's Complement

Subtraction is converted to addition: $A - B = A + (2\text{'s complement of } B)$.

This means the CPU does not need a separate subtractor circuit — the ALU reuses the adder by complementing the subtrahend and asserting carry-in = 1.

Example: $15 - 9 = 6$

- 15 in binary (8-bit): 00001111
- 9 in binary: $00001001 \rightarrow 2\text{'s complement of } 9 = 11110111$
- $00001111 + 11110111 = 100000110 \rightarrow$ Discard carry: $00000110 = 6$ ✓

3.8.4 Overflow Detection

Overflow occurs when the result of an arithmetic operation exceeds the range representable in the given number of bits. For signed addition:

- Positive + Positive = Negative result $\rightarrow$ Overflow
- Negative + Negative = Positive result $\rightarrow$ Overflow

Hardware overflow detection rule: Overflow occurs if and only if the carry into the sign bit (MSB) does NOT equal the carry out of the sign bit.

- Equivalently: $V = \text{Carry-in to MSB XOR Carry-out from MSB}$

Q Analysing Misconceptions: 2's Complement and Subtraction Hardware

✗ MYTH: Computers use separate adder and subtractor circuits for addition and subtraction operations.

✓ REALITY: Modern CPUs use a single Arithmetic Logic Unit (ALU) that performs subtraction by computing $A + (\text{NOT } B) + 1$, which is $A + (\text{2's complement of } B)$. A multiplexer selects whether to complement B based on whether the operation is add or subtract. This elegant reuse of hardware is one of the key reasons 2's complement is the universal standard.

3.9 Fast Adder Designs — Carry Look-Ahead Adder

Adding two n-bit numbers seems straightforward when using full adders chained together — each bit's carry is computed and passed to the next stage. This design is called a Ripple Carry Adder (RCA). However, the carry must ripple through all n stages sequentially, creating a critical performance bottleneck for wide operands (32-bit, 64-bit). The Carry Look-Ahead Adder (CLA) was developed to overcome this limitation.

3.9.1 Ripple Carry Adder — The Limitation

In an n-bit ripple carry adder, bit i cannot produce its final output until bit i−1 has computed its carry-out. For a 64-bit adder, the carry may need to propagate through all 64 full adder stages in the worst case.

- Worst case delay: $n \times (\text{full adder delay})$. For $n = 64$, this is 64 gate delays just for carry propagation.
- As processor clock frequencies have risen to 3–5 GHz, even a few extra gate delays in the critical path can limit maximum frequency.

3.9.2 Generate and Propagate Signals

The CLA insight comes from defining two signals for each bit position i :

- Generate (G_i): $G_i = A_i \text{ AND } B_i$. This bit position will produce a carry-out regardless of the carry-in. If both A_i and B_i are 1, a carry is generated.
- Propagate (P_i): $P_i = A_i \text{ OR } B_i$ (or $A_i \text{ XOR } B_i$). This bit position will pass a carry-in to the carry-out. If at least one of A_i , B_i is 1, an incoming carry is propagated forward.

The carry-out at bit position i is then:

- $C_{i+1} = G_i + (P_i \cdot C_i)$

Expanding for the first four bits (starting from $C_0 = C_{in}$):

- $C_1 = G_0 + P_0 \cdot C_0$
- $C_2 = G_1 + P_1 \cdot C_1 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$
- $C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$
- $C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$

All four carry signals are computed in parallel — each requiring only 2 gate delays (AND then OR), regardless of how many bits are involved. This is the fundamental advantage of CLA.

3.9.3 CLA Performance Comparison

Adder Type	Delay for 64-bit Addition
Ripple Carry Adder (RCA)	~64 full adder delays (sequential carry chain)
4-bit CLA Block	2 gate delays for all carries within the block
16-bit CLA (4 × 4-bit blocks)	4 gate delays total (2 levels of look-ahead)
64-bit CLA (hierarchical)	6–8 gate delays — major performance gain

3.9.4 Real-World Application

Modern ALUs use CLA or its variations (prefix adders, Han-Carlson, Kogge-Stone) extensively. For example:

- ARM Cortex-A78 ALU (used in Snapdragon 888, which powers Samsung Galaxy S21 and OnePlus 9) uses a 64-bit Kogge-Stone adder — a parallel prefix form of CLA that achieves $O(\log n)$ delay.
- RISC-V processors developed by IIT Madras's Shakti project also employ CLA-based designs for their integer ALUs.

💡 Did You Know?

The concept of carry look-ahead was first described by Gerald Weinberger and J.L. Smith at IBM in 1958. It remains one of the most impactful arithmetic circuit innovations in the history of computing, forming the core of virtually every modern processor's ALU — from microcontrollers in household appliances to supercomputer nodes.

✔ Let's Check

Q.No	Type	Question
1	MCQ	The Generate signal G_i in a CLA adder is defined as: (A) $A_i \text{ XOR } B_i$ (B) $A_i \text{ OR } B_i$ (C) $A_i \text{ AND } B_i$ (D) $\text{NOT}(A_i \text{ AND } B_i)$
2	MCQ	What is the 8-bit 2's complement representation of -12 ? (A) 00001100 (B) 11110011 (C) 11110100 (D) 10001100
3	Fill in Blank	In a ripple carry adder, the worst-case delay is proportional to _____ (number of bit stages).
4	Fill in Blank	Overflow in signed addition occurs when the carry into the MSB _____ the carry out of the MSB.
5	True/False	Subtraction of two signed numbers can be performed using an adder with 2's complement of the subtrahend. (True/False)
6	True/False	A carry look-ahead adder computes carry signals sequentially, one after another. (True/False)

Answers:

Q.No	Answer
1	(C) $A_i \text{ AND } B_i$
2	(C) 11110100 ($12 = 00001100 \rightarrow \text{invert} = 11110011 \rightarrow +1 = 11110100$)
3	n (number of bit stages)
4	does NOT equal (differs from)
5	True
6	False — CLA computes all carry signals simultaneously in parallel, which is its primary advantage over the ripple carry adder.

Case Study

Case Study: Cache Hierarchy Optimisation in ISRO's NavIC Signal Processing System

Background

India's Navigation with Indian Constellation (NavIC) system, launched progressively between 2016 and 2018, provides positioning, navigation, and timing (PNT) services across the Indian subcontinent. The signal processing units onboard the NavIC satellites and in ground control stations (located in Byalalu, Karnataka) must perform intensive real-time arithmetic: carrier phase tracking, pseudorange computation, and clock correction algorithms. These involve repeated addition and multiplication of 32-bit and 64-bit signed integers at processing rates exceeding 50 MHz.

The Problem

ISRO's Space Applications Centre (SAC, Ahmedabad) identified a critical performance bottleneck in the ground station's signal correlator hardware (circa 2019). The correlator used a custom FPGA-based processor with a flat memory architecture — no cache hierarchy. Every arithmetic operation involving navigation constants (stored in lookup tables) required a full DRAM access (~80 ns). At a 50 MHz clock rate, this meant most processing cycles were stalled waiting for memory, achieving an effective utilisation of barely 35%.

Analysis

Engineers analysed the memory access patterns of the correlator code. Findings:

- Navigation constants (ephemeris data, satellite coefficients) were reused over many clock cycles — high temporal locality.
- Lookup table scans were sequential — high spatial locality suited to 64-byte cache lines.
- Arithmetic-intensive inner loops repeatedly accessed the same 4–8 KB of data.

Solution

A two-level cache was implemented in the updated FPGA design (2021): a 32 KB direct-mapped L1 instruction cache and a 128 KB 4-way set-associative L2 data cache with LRU replacement. The arithmetic units were upgraded to use a 32-bit CLA-based adder (4×8 -bit CLA blocks) achieving carry computation in 3 gate delays vs the previous 32-stage ripple carry. Virtual address translation was handled via a lightweight TLB with 64 entries covering the 512 KB working set.

Results

- Cache hit rate achieved: 94.7% (L1 + L2 combined), reducing average memory access time from 80 ns to under 8 ns.
- Processor utilisation improved from 35% to 87%.

- Correlation throughput increased by $2.8\times$ without increasing clock frequency or power consumption.
- The upgraded system is now deployed at the ISRO Master Control Facility, Hassan, Karnataka.

Reference: ISRO Technical Report — NavIC Ground Segment Upgrade, SAC-2021-003; ISRO Annual Report 2021–22, Department of Space, Government of India.

Summary

Module 3 covered two fundamental pillars of computer architecture: the memory subsystem and arithmetic hardware. Together, these components determine how efficiently a computer system can store, retrieve, and process data.

Memory Hierarchy and Cache:

- Memory is organised in a hierarchy — from ultra-fast, small SRAM caches to large, slow DRAM and disk storage. The principle of locality (temporal and spatial) makes this hierarchy effective.
- Cache memory bridges the speed gap between the CPU and main memory. Key metrics include hit rate, miss rate, and effective access time.
- Three mapping techniques govern where data can be stored in cache: direct-mapped (simple, conflict-prone), fully associative (flexible, expensive), and set-associative (the practical compromise used in modern processors).
- Replacement algorithms — LRU, FIFO, LFU, and random — determine which cache block is evicted when new data must be loaded.
- Write-back and write-through policies manage consistency between cache and main memory, with write-back being more efficient but complex.
- The three Cs of cache misses — Compulsory, Capacity, and Conflict — provide a framework for analysing and reducing miss rates.

Virtual Memory:

- Virtual memory abstracts physical RAM, providing each process with its own address space. Paging divides memory into fixed-size pages/frames.
- The MMU translates virtual addresses to physical addresses using the page table. A page fault triggers OS intervention to load data from disk.

- The TLB caches recent address translations, dramatically reducing translation overhead (typically achieving 99%+ hit rates).

Arithmetic Hardware:

- All digital circuits are built from logic gates (AND, OR, NOT, XOR, etc.). Flip-flops (D, SR, JK) are the fundamental storage elements in sequential circuits.
- 2's complement is the universal standard for signed integer representation, enabling subtraction to be performed using an adder circuit.
- Overflow detection in signed arithmetic uses the carry-in/carry-out relationship at the MSB.
- The Ripple Carry Adder is simple but slow — carry propagates sequentially through all bit stages.
- The Carry Look-Ahead Adder computes all carry signals in parallel using Generate (Gi) and Propagate (Pi) signals, reducing carry delay to $O(\log n)$ and enabling high-speed processor ALUs.

Glossary

Term	Definition
Cache Hit	A memory access where the requested data is found in the cache, avoiding a slower main memory access.
Cache Line	The fundamental unit of data transfer between cache and main memory, typically 64 bytes.
Direct Mapping	A cache mapping technique where each main memory block maps to exactly one cache line, determined by modulo arithmetic.
Flip-Flop	A bistable sequential circuit that can store one bit of data and changes state on a clock signal edge.
2's Complement	A binary number representation system for signed integers obtained by inverting all bits and adding 1; enables subtraction using adder circuits.
LRU	Least Recently Used — a cache/page replacement policy that evicts the block not accessed for the longest period.
Page Fault	An exception raised when a process accesses a virtual page not currently resident in physical RAM, triggering OS intervention to load it from disk.
TLB	Translation Lookaside Buffer — a fast hardware cache that stores recent virtual-to-physical address translations to speed up memory access.
Carry Look-Ahead Adder	A fast adder design that computes all carry signals simultaneously using Generate and Propagate signals, achieving $O(\log n)$ carry delay.
Set-Associative Cache	A cache organisation where main memory blocks map to a specific set but can occupy any line within that set, balancing flexibility and hardware cost.

Self Assessment Questions

Section A: Multiple Choice Questions (MCQ)

1. The principle of locality used to justify the memory hierarchy is:
 - A) Temporal locality only
 - B) Spatial locality only
 - C) Both temporal and spatial locality
 - D) Neither — memory hierarchy uses random access patterns
2. In a direct-mapped cache with 32 lines, memory block number 98 maps to which cache line?
 - A) Line 2
 - B) Line 3
 - C) Line 4
 - D) Line 32
3. Which cache mapping technique requires parallel comparators for every cache line?
 - A) Direct Mapped
 - B) 2-Way Set Associative
 - C) 4-Way Set Associative
 - D) Fully Associative
4. A 4-way set-associative cache has 8 sets. The total number of cache lines is:
 - A) 8
 - B) 16
 - C) 32
 - D) 4
5. The LRU replacement algorithm is based on which principle?
 - A) The most recently used block will be needed again soon
 - B) The least recently used block is least likely to be needed in the future

-
- C) The block loaded first should be evicted first
 - D) A random block is selected for eviction
6. Belady's anomaly (increasing misses with more cache) can occur with which replacement algorithm?
- A) LRU
 - B) Optimal (OPT)
 - C) FIFO
 - D) LFU
7. In write-through cache policy:
- A) Writes are made only to cache and main memory is updated only at eviction
 - B) Every write to cache is immediately written to main memory as well
 - C) Writes bypass the cache entirely and go directly to main memory
 - D) Writes are buffered and committed in batches
8. Which type of cache miss occurs when first accessing data that has never been in cache?
- A) Conflict miss
 - B) Capacity miss
 - C) Coherence miss
 - D) Compulsory miss
9. Virtual memory allows programs to:
- A) Execute faster than without virtual memory
 - B) Use more memory than physically available in RAM
 - C) Eliminate the need for a page table
 - D) Access hardware registers directly
10. A page fault occurs when:
- A) A page in the TLB has an invalid tag
 - B) The program accesses a virtual page not currently in physical RAM
-

- C) Two virtual pages map to the same physical frame
 - D) The cache is full
11. The Translation Lookaside Buffer (TLB) is:
- A) A section of main memory for storing page tables
 - B) A disk cache for virtual memory pages
 - C) A fast hardware cache for recent virtual-to-physical address translations
 - D) A register in the CPU that holds the current page table base address
12. For n-bit 2's complement representation, the minimum (most negative) value is:
- A) $-(2^n - 1)$
 - B) $-2^{(n-1)}$
 - C) $-(2^n)$
 - D) $-(2^{(n-1)} - 1)$
13. The XOR gate is used in a half adder to compute the:
- A) Carry bit
 - B) Sum bit
 - C) Borrow bit
 - D) Overflow bit
14. In a D flip-flop, the output Q equals:
- A) The complement of input D at the clock edge
 - B) The value of D at the clock edge
 - C) 1 whenever $D=1$ and $\text{clock}=1$
 - D) The AND of D and the previous state of Q
15. The condition $J=K=1$ in a JK flip-flop causes:
- A) The output to be set to 1
 - B) The output to be reset to 0
 - C) No change in output
 - D) The output to toggle

16. Overflow in signed 2's complement addition occurs when:

- A) The carry out of the MSB is 1
- B) The result requires more bits than available
- C) Carry into MSB does NOT equal carry out of MSB
- D) Both operands are negative

17. In an n-bit ripple carry adder, the worst-case carry propagation delay is proportional to:

- A) $\log_2(n)$
- B) n
- C) n^2
- D) constant (independent of n)

18. The Generate signal G_i in a carry look-ahead adder is defined as:

- A) $G_i = A_i \text{ XOR } B_i$
- B) $G_i = A_i \text{ OR } B_i$
- C) $G_i = A_i \text{ AND } B_i$
- D) $G_i = \text{NOT}(A_i \text{ AND } B_i)$

19. A carry look-ahead adder computes carry signals:

- A) Sequentially, one bit at a time
- B) In parallel for all bit positions simultaneously
- C) Only for even-numbered bit positions
- D) Using a feedback loop from the MSB

20. Which of the following is a universal logic gate (can implement all Boolean functions)?

- A) AND
- B) OR
- C) XOR
- D) NAND

Section B: True / False

State whether the following statements are TRUE or FALSE.

1. A cache hit rate of 95% means the processor accesses main memory only 5% of the time.
2. In a set-associative cache, increasing the number of ways reduces conflict misses.
3. The page table is stored in the TLB for faster access during every memory operation.
4. Subtraction of signed binary numbers can be performed using a 2's complement adder.
5. The carry look-ahead adder has a constant delay regardless of the number of bits.

Section C: Short Answer Questions

Answer each question in 3–5 sentences.

1. Explain the difference between direct-mapped and set-associative cache. Why is set-associative cache preferred in modern processors?
2. What is a TLB and why is it needed in a virtual memory system?
3. Describe the LRU cache replacement algorithm with a brief example showing how it works.
4. Convert -45 to its 8-bit 2's complement representation and verify by adding it to $+45$.
5. What are the Generate (G) and Propagate (P) signals in a carry look-ahead adder? How do they help eliminate carry delay?

Self Assessment: Answer Keys

MCQ

Q.No	Correct Answer & Explanation
1	C) Both temporal and spatial locality — Memory hierarchy exploits both: temporal (recently accessed data used again) and spatial (nearby data accessed soon).
2	A) Line 2 — $98 \bmod 32 = 2$. The line number is the remainder when dividing the block number by the total cache lines.
3	D) Fully Associative — requires a comparator for each cache line to search all simultaneously.
4	C) 32 — Total lines = sets $\times$ ways = $8 \times 4 = 32$.
5	B) The least recently used block is least likely to be needed in the future — based on temporal locality.
6	C) FIFO — Belady's anomaly uniquely affects FIFO; LRU and OPT are immune.
7	B) Every write to cache is immediately written to main memory as well — write-through maintains consistency at the cost of bus traffic.
8	D) Compulsory miss — also called a cold miss; occurs the very first time a block is accessed.
9	B) Use more memory than physically available — virtual memory uses disk as an extension of RAM.
10	B) The program accesses a virtual page not currently in physical RAM — causing the OS to load it from disk.
11	C) A fast hardware cache for recent virtual-to-physical address translations — dramatically reduces page table lookup time.

12	B) $-2^{(n-1)}$ — For 8-bit: -128. The MSB represents a negative weight of $2^{(n-1)}$.
13	B) Sum bit — $\text{Sum} = A \text{ XOR } B$. The carry bit is computed by the AND gate: $\text{Carry} = A \text{ AND } B$.
14	B) The value of D at the clock edge — D flip-flop captures D on the active clock edge.
15	D) The output to toggle — $J=K=1$ inverts the current state of Q; this is unique to the JK flip-flop.
16	C) Carry into MSB does NOT equal carry out of MSB — $V = C_{in} \text{ XOR } C_{out}$ of the sign bit.
17	B) n — carry ripples through n stages sequentially in the worst case.
18	C) $G_i = A_i \text{ AND } B_i$ — a carry is generated when both bits are 1.
19	B) In parallel for all bit positions simultaneously — this is the key advantage of CLA over ripple carry.
20	D) NAND — NAND is a universal gate; all Boolean functions can be implemented using only NAND gates.

True / False Answer Key

Q.No	Answer / Explanation
1	TRUE — Hit rate 95% means 5% of accesses miss the cache and require main memory access.
2	TRUE — More ways means a block can reside in more locations within its set, reducing the chance of two blocks conflicting.

3	FALSE — The page table is stored in main memory (RAM). The TLB is a separate hardware cache that stores a small number of recent page table entries to speed up translation.
4	TRUE — $A - B = A + (2\text{'s complement of } B)$. The same adder hardware is used with the subtrahend complemented and carry-in set to 1.
5	FALSE — CLA reduces carry delay to $O(\log n)$, not constant. For very wide adders (e.g., 64-bit), hierarchical CLA is used, but delay still grows (slowly) with the number of bits.

Short Answer Key

Q.No	Model Answer
1	Direct-mapped cache maps each memory block to exactly one cache line, making lookup fast but causing conflict misses when blocks compete for the same line. Set-associative divides the cache into sets, allowing a block to occupy any line within its assigned set (k choices). Set-associative is preferred because it reduces conflict misses while maintaining fast lookup using a small associative search — most modern L1/L2/L3 caches use 4-way or 8-way set associativity.
2	The TLB (Translation Lookaside Buffer) is a small, fast hardware cache inside the MMU that stores recent virtual-to-physical page address translations. It is needed because the page table is stored in main memory — looking up every translation in main memory would double or triple memory access time. With a TLB hit rate of ~99%, most translations are resolved in 1–2 cycles, making virtual memory practical without significant performance overhead.

3	LRU (Least Recently Used) evicts the cache block that has not been accessed for the longest time, exploiting temporal locality. Example with 3-slot cache: access sequence A, B, C (fills cache), then D → evict A (oldest). Access A → evict B. Access E → evict C. LRU maintains a 'use order' so the least recently used item is always identified for eviction.
4	+45 = 00101101. To get -45: invert → 11010010, add 1 → 11010011. Verification: 00101101 (45) + 11010011 (-45) = 100000000. Discard 9th bit carry → 00000000 = 0 ✓. The result confirms that +45 + (-45) = 0.
5	Generate ($G_i = A_i \text{ AND } B_i$): bit position i will produce a carry regardless of carry-in (both inputs are 1). Propagate ($P_i = A_i \text{ OR } B_i$): bit i will pass on any incoming carry to the next position. Using these signals, all carry signals are computed in parallel: $C_{i+1} = G_i + P_i \cdot C_i$. This eliminates the need to wait for each stage's carry sequentially, reducing carry delay from $O(n)$ in a ripple carry adder to $O(\log n)$ in CLA.

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA

